

CSCE 611 OPERATING SYSTEMS · MODULE 07 · LECTURE NOTES

Virtual Memory and Paging Policy

The kernel side of the mechanism CSCE 614 described.

MODULE OUTCOMES

- Implement demand paging via the page fault handler.
- Implement copy-on-write and lazy allocation.
- Compare page replacement policies against the optimal.
- Explain thrashing and the working set model.
- Explain mmap and the unification of files and memory.

1 The page fault handler

CSCE 614 Module 07 covered the hardware: page tables, the TLB, and address translation. This module covers what happens when translation *fails* — and that turns out to be the mechanism behind most of modern virtual memory.

Fault cause	Kernel response	Feature it implements
Address is valid but no page has ever been allocated.	Allocate a zeroed physical page and map it.	Lazy allocation. malloc and sbrk return instantly; physical memory is committed only on first touch.
Write to a page marked read-only and flagged copy-on-write.	Allocate a new page, copy the contents, map it writable, decrement the old page's reference count.	Copy-on-write fork (Module 02).
Page is valid but has been evicted to backing store.	Find a free frame (evicting if necessary), read the page in, update the table.	Demand paging and memory over-subscription.
Page belongs to a mapped file and is not resident.	Read the corresponding file block into a frame and map it.	mmap , and executable loading.
Address is genuinely not mapped, or permissions forbid the access.	Deliver SIGSEGV.	Catching program bugs.

```

void pagefault(uint64 va, int cause) {
    struct vma *v = find_vma(myproc(), va);
    if (!v) return kill(SIGSEGV);
    if (cause == WRITE && !(v->perm & PERM_W)) return kill(SIGSEGV);

    if (cause == WRITE && is_cow(va)) copy_page(va);
    else if (v->file) read_page_from_file(v, va);
    else if (swapped_out(va)) read_page_from_swap(va);
    else map_zero_page(va);

    // On return, the hardware RE-EXECUTES the faulting instruction.
}

```

RETRY IS WHAT MAKES IT TRANSPARENT

When the handler returns, the hardware restarts the instruction that faulted. This time the translation succeeds and the instruction completes normally.

The process has no way to observe that anything happened — no signal, no return code, no change in its registers. The only evidence is that the instruction took longer.

That is why the kernel can implement lazy allocation, copy-on-write, paging, and file mapping without any cooperation from the program, and it is an unusually economical piece of design: one hardware mechanism, five quite different features.

2 Page replacement

Physical memory fills. To bring a page in, one must go out. Which?

Policy	Rule	Assessment
OPT	Evict the page whose next use is furthest in the future.	Provably optimal and unimplementable — it requires the future. Its role is as a yardstick: simulate it offline from a trace and measure how close your real policy comes. Exactly the role SJF plays in Module 03.
FIFO	Evict the page that has been resident longest.	Trivial to implement and genuinely bad: residence time has no relationship to future usefulness. Suffers Belady's anomaly.
LRU	Evict the least recently used page.	An excellent predictor, because of locality. Exact LRU requires updating a timestamp or list position on <i>every</i> memory access, which is far too expensive.
Clock	Approximate LRU using a hardware reference bit and a circular scan.	What real kernels use. Nearly LRU quality at nearly no cost.
Random	Evict a page at random.	Noticeably worse than LRU and noticeably better than one expects. Its merit is that it has no pathological cases.

BELADY'S ANOMALY

It seems self-evident that giving a system more memory cannot increase its fault count. For FIFO, that is false: reference strings exist where increasing the number of frames *increases* the number of faults.

The cause is that FIFO's eviction order is unrelated to usefulness, so a larger pool can hold the wrong pages for longer.

Policies with the **stack property** cannot do this: the set of pages resident with n frames is always a subset of the set resident with $n+1$ frames, so more memory can only help. LRU and OPT have this property; FIFO does not.

Beyond being a curiosity, it is a useful reminder that plausible monotonicity assumptions need proof.

2.1 Clock

```
int clock_evict(void) {
    for (;;) {
        if (!frames[hand].referenced)
            return hand;           // untouched since last sweep
        frames[hand].referenced = 0; // second chance
        hand = (hand + 1) % nframes;
    }
}
```

The hardware sets a **reference bit** in the page table entry whenever a page is accessed — one bit, set automatically, at no software cost. The kernel clears it. A page is evicted only if the clock hand reaches it and finds the bit still clear, meaning it has not been touched in a full sweep.

The division of labour is the point: the hardware does the per-access work (setting one bit), and the kernel does the occasional work (sweeping). Adding a second bit — the dirty bit — gives the four-class refinement that prefers evicting clean pages, since those need no write-back.

3 Thrashing and the working set

THRASHING IS A CLIFF WITH POSITIVE FEEDBACK

As more processes are admitted, throughput rises — until the sum of their active working sets exceeds physical memory.

At that point each process faults almost immediately after being scheduled, and each fault evicts a page that some other process is about to need. Throughput does not degrade gracefully; it **collapses**. The CPU sits idle while the disk saturates.

The feedback makes it worse: a scheduler observing idle CPU concludes the system is underloaded and admits more processes, deepening the problem.

The classical fix is to reduce the degree of multiprogramming — swap whole processes out so that the remainder's working sets fit. Modern systems largely rely on the OOM killer instead, which is blunter and arguably more honest: if the workload does not fit, something has to go.

$$w(t, \tau) = \{ \text{pages referenced in the interval } (t - \tau, t] \}$$

Denning's **working set** model formalises this. A process executes efficiently when its working set is resident and thrashes when it is not, so the admission rule is to keep the sum of resident working sets below physical memory.

This is the locality argument of CSCE 614 Module 05 at a coarser granularity. Paging is caching, with main memory as the cache and the disk as the backing store, and the same concepts transfer: working set, capacity miss, replacement policy, and a sharp cliff when the working set exceeds capacity. Only the numbers change — a cache miss costs 250 cycles, a page fault costs a hundred thousand.

4 mmap

mmap maps a file into a process's address space. The file is then accessed with ordinary loads and stores, with pages brought in on demand by the fault handler.

mmap	read/write
No copy: the page cache pages <i>are</i> your memory.	Data is copied from the page cache into your buffer.
Random access is natural — just index.	Natural for sequential streaming; random access needs lseek.
Costs are invisible: a page fault looks like an ordinary memory access that happened to take a millisecond.	Costs are explicit and easy to reason about.
An I/O error arrives as SIGBUS, not as a return value — awkward to handle.	Errors are return values you can check.
Limited by address space and by the cost of setting up mappings.	No such limit.

mmap	read/write
Shared mappings give processes shared memory backed by a file.	Sharing requires explicit IPC.

Two things worth knowing. First, the **page cache serves both** `read()` and `mmap`, which is why the two views of a file are coherent — a write through one is visible through the other. Second, **this is how executables are loaded**: the kernel maps the binary's text and data segments and lets demand paging bring in code as it is first executed, which is why starting a large program does not read the whole file.

WHERE CSCE 614 AND THIS COURSE JOIN

CSCE 614 Module 07 described the hardware: multi-level page tables, the TLB, translation cost, huge pages, TLB reach.

This module describes the software on the other side of the fault: what the kernel does when the hardware cannot complete a translation, and what it chooses to evict when memory runs out.

Together they form one mechanism. **Hardware detects and reports; the kernel decides and repairs; hardware retries.** Lazy allocation, copy-on-write, demand paging, memory-mapped files, and executable loading are all the same fault, handled five different ways.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OSTEP — Chapters 21–24 (Beyond Physical Memory, Swapping, Policies)	Replacement policies with simulations, Belady's anomaly, and the working set model.
MIT 6.1810 — page tables lecture and the COW and lazy allocation labs	The labs are precisely Project 2's requirements.
xv6 book — Chapter 3 (Page tables)	The implementation, with the fault path through real code.
Denning — The Working Set Model for Program Behavior (free)	The original working set paper. Still the clearest statement of why locality makes paging work at all.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement lazy allocation in xv6: make `sbrk` only adjust the size, and allocate pages in the fault handler. Verify that allocating 1 GB and touching one page uses one page of physical memory.
- 2 Implement copy-on-write fork with page reference counting. Fork a process with a large heap and show that physical memory does not double until writes occur.
- 3 Implement `mmap` and `munmap` for file-backed mappings in xv6.
- 4 Simulate FIFO, LRU, Clock, Random, and OPT against the same reference string. Plot fault count against frame count for each.
- 5 Construct a reference string exhibiting Belady's anomaly for FIFO. Verify that LRU does not show it on the same string.
- 6 Implement demand paging with a backing store and a replacement policy of your choice. Plot fault count against working-set size and identify the cliff.
- 7 Induce thrashing deliberately: run several processes whose combined working sets exceed physical memory. Measure CPU utilisation and disk activity as it collapses.
- 8 Compare `mmap` against read for sequential and random access to a large file. Explain both results.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name four page faults that are not errors and say what feature each implements.
- 2 Why is the faulting instruction retried, and what does that make possible?
- 3 What role does OPT play, given that it cannot be implemented?
- 4 Explain Belady's anomaly and say why LRU is immune to it.
- 5 How does Clock approximate LRU, and what is the division of labour between hardware and kernel?
- 6 Why does thrashing collapse rather than degrade, and what makes the feedback vicious?
- 7 Give two advantages and two disadvantages of `mmap` over read.